

```
In [ ]: import marimo as mo
import torch
import matplotlib.pyplot as plt
```

Install VOC2012 dataset on your laptop (You may copy the dataset from me or Siddharth on your pendrive). Use YOLOv8 to train and identify objects in this dataset. Describe the results of your evaluation.

Load the MUTAG dataset using PyTorch Geometric. Visualize the molecular graphs for at least 4 sample molecules (both mutagenic and non-mutagenic examples). Design and train a Graph Neural Network (GNN) of your choice (e.g. GCN, GAT, GraphSAGE, GIN) to classify molecules in the MUTAG dataset as mutagenic or non-mutagenic.

```
In [ ]: from torch_geometric.datasets import TUDataset

#load the dset
d=TUDataset(root='data/TUDataset', name='MUTAG')
torch.manual_seed(42)
dset=d.shuffle()
#basic inspection
print(f"no. of graphs: {len(dset)}\n no. of node features: {dset.num_features}")
print(dset[0])
```

```
no. of graphs: 188
no. of node features: 7
no. of classes: 2
Data(edge_index=[2, 44], x=[21, 7], edge_attr=[44, 4], y=[1])
```

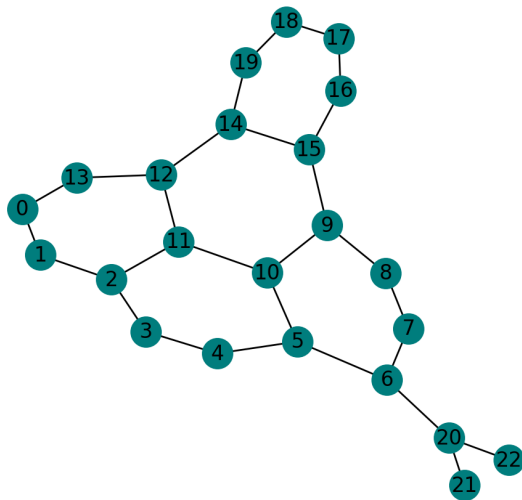
```
In [ ]: # visualize
import networkx as nx
from torch_geometric.utils import to_networkx

mutagenic=[data for data in dset if data.y.item() == 1]
non_mutagenic = [data for data in dset if data.y.item() == 0]

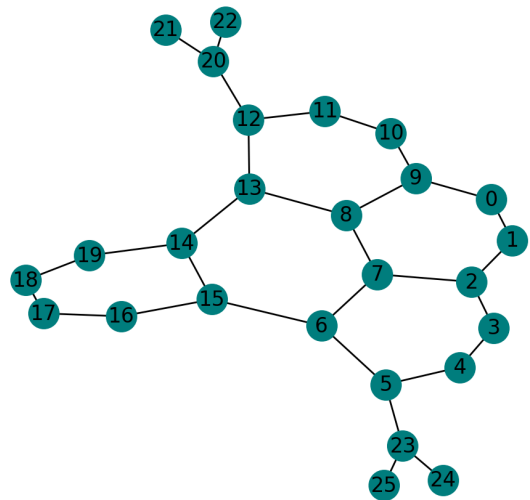
#selecting class
samples = [
    (mutagenic[6], "Mutagenic"),
    (mutagenic[7], "Mutagenic"),
    (non_mutagenic[6], "Non-mutagenic"),
    (non_mutagenic[7], "Non-mutagenic"),
]

#plot
plt.figure(figsize=(10,10))
for i, (data, label) in enumerate(samples):
    graph=to_networkx(data, to_undirected=True)
    plt.subplot(2,2,i+1)
    nx.draw(graph, with_labels=True, node_color='teal')
    plt.title(f"{label} (graph{i})")
plt.tight_layout()
plt.show()
```

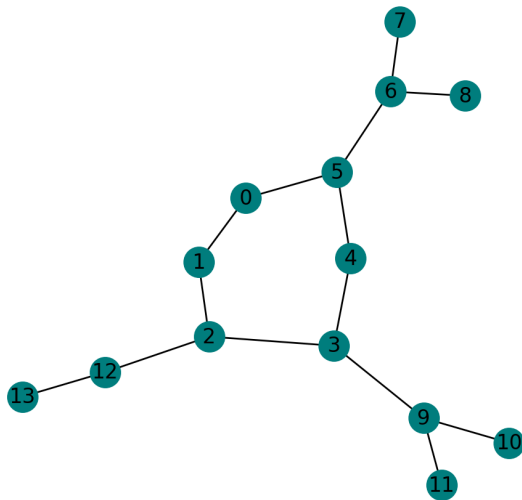
Mutagenic (graph0)



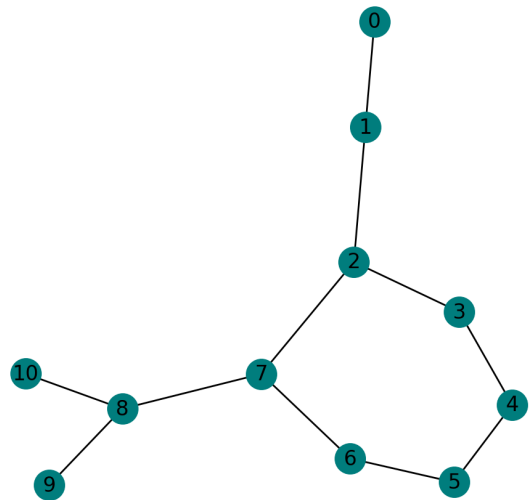
Mutagenic (graph1)



Non-mutagenic (graph2)



Non-mutagenic (graph3)



```
In [ ]: # training a GNCh
        torch.manual_seed(67)
        train_dset=dset[:150]
        test_dset=dset[150:]
        print(len(train_dset), len(test_dset))
```

150 38

### Batching Graphs

```
In [ ]: from torch_geometric.loader import DataLoader

        train_loader=Dataloader(train_dset, batch_size=32, shuffle=True)
        test_loader=Dataloader(test_dset, batch_size=32)

        #inspect one batch
        for batch in train_loader:
            print(batch)
            break
```

```
DataBatch(edge_index=[2, 1312], x=[594, 7], edge_attr=[1312, 4], y=[32], batch=[594], ptr=[33])
```

```
In [ ]: import torch.nn.functional as F
        from torch_geometric.nn import GCNConv, global_mean_pool

        # initialize layers
        conv1 = GCNConv(dset.num_features, 32)
        conv2 = GCNConv(32, 32)
        lin = torch.nn.Linear(32, 2)

        # move to device
        device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        conv1, conv2, lin = conv1.to(device), conv2.to(device), lin.to(device)

        def forward(x, edge_index, batch, training=False):

            # L1
            h1 = conv1(x, edge_index)
            h2 = F.relu(h1)
            h2 = F.dropout(h2, p=0.3, training=training)

            # L2
            h3 = conv2(h2, edge_index)
            h4 = F.relu(h3)
            h4 = F.dropout(h4, p=0.3, training=training)

            # Pool
            h5 = global_mean_pool(h4, batch)

            # Output
            out = lin(h5)

            return out
```

```
In [ ]: # optimizer + loss
        params = list(conv1.parameters()) + list(conv2.parameters()) + list(lin.parameters())
        optimizer=torch.optim.Adam(params, lr=0.005, weight_decay=5e-4)
        criterion=torch.nn.CrossEntropyLoss()
```

```
In [ ]: # training function
        def train():
            total_loss = 0
            conv1.train(); conv2.train(); lin.train()
            for data in train_loader:
                data = data.to(device)
                optimizer.zero_grad()
                out = forward(data.x, data.edge_index, data.batch, training=True)
                loss = criterion(out, data.y)
                loss.backward()
                optimizer.step()
                total_loss += loss.item()

            return total_loss / len(train_loader)
```

```

In [ ]: #evaluation
def test(loader):
    conv1.eval(); conv2.eval(); lin.eval()
    correct=0
    with torch.no_grad():
        for data in loader:
            data=data.to(device)
            out=forward(data.x, data.edge_index, data.batch)
            pred=out.argmax(dim=1)
            correct+=(pred == data.y).sum().item()

    return correct / len(loader.dataset)
#training loop
best_test_acc = 0

for epoch in range(1, 41):
    loss = train()
    train_acc = test(train_loader)
    test_acc = test(test_loader)

    if test_acc > best_test_acc:
        best_test_acc = test_acc

    print(f"Epoch {epoch:03d}, Loss: {loss:.4f}, Train Acc: {train_acc:.4f},

print(f"\n #Best Test Accuracy: {best_test_acc:.4f}")

```

Epoch 001, Loss: 0.6637, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 002, Loss: 0.6346, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 003, Loss: 0.6115, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 004, Loss: 0.6096, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 005, Loss: 0.5968, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 006, Loss: 0.5935, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 007, Loss: 0.6108, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 008, Loss: 0.5974, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 009, Loss: 0.5878, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 010, Loss: 0.5771, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 011, Loss: 0.5717, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 012, Loss: 0.5700, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 013, Loss: 0.5654, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 014, Loss: 0.5615, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 015, Loss: 0.5621, Train Acc: 0.6933, Test\_Acc: 0.5526  
Epoch 016, Loss: 0.5499, Train Acc: 0.7000, Test\_Acc: 0.5789  
Epoch 017, Loss: 0.5421, Train Acc: 0.7000, Test\_Acc: 0.5789  
Epoch 018, Loss: 0.5405, Train Acc: 0.7133, Test\_Acc: 0.5789  
Epoch 019, Loss: 0.5278, Train Acc: 0.7467, Test\_Acc: 0.6053  
Epoch 020, Loss: 0.5235, Train Acc: 0.7467, Test\_Acc: 0.6316  
Epoch 021, Loss: 0.5328, Train Acc: 0.7733, Test\_Acc: 0.6579  
Epoch 022, Loss: 0.5122, Train Acc: 0.7467, Test\_Acc: 0.6579  
Epoch 023, Loss: 0.5259, Train Acc: 0.7733, Test\_Acc: 0.6842  
Epoch 024, Loss: 0.5146, Train Acc: 0.7533, Test\_Acc: 0.6316  
Epoch 025, Loss: 0.5047, Train Acc: 0.7467, Test\_Acc: 0.6579  
Epoch 026, Loss: 0.5065, Train Acc: 0.7600, Test\_Acc: 0.6842  
Epoch 027, Loss: 0.4979, Train Acc: 0.7667, Test\_Acc: 0.6579  
Epoch 028, Loss: 0.5025, Train Acc: 0.7533, Test\_Acc: 0.6579  
Epoch 029, Loss: 0.4870, Train Acc: 0.7467, Test\_Acc: 0.6579  
Epoch 030, Loss: 0.4900, Train Acc: 0.7533, Test\_Acc: 0.6579  
Epoch 031, Loss: 0.5018, Train Acc: 0.7667, Test\_Acc: 0.6316  
Epoch 032, Loss: 0.4994, Train Acc: 0.7533, Test\_Acc: 0.6579  
Epoch 033, Loss: 0.4975, Train Acc: 0.7733, Test\_Acc: 0.6842  
Epoch 034, Loss: 0.5159, Train Acc: 0.7733, Test\_Acc: 0.6842  
Epoch 035, Loss: 0.5067, Train Acc: 0.7800, Test\_Acc: 0.6842  
Epoch 036, Loss: 0.4956, Train Acc: 0.7800, Test\_Acc: 0.6842  
Epoch 037, Loss: 0.4917, Train Acc: 0.7800, Test\_Acc: 0.6842  
Epoch 038, Loss: 0.4968, Train Acc: 0.7800, Test\_Acc: 0.6842  
Epoch 039, Loss: 0.5068, Train Acc: 0.7800, Test\_Acc: 0.6842  
Epoch 040, Loss: 0.4871, Train Acc: 0.7667, Test\_Acc: 0.6842

#Best Test Accuracy: 0.6842